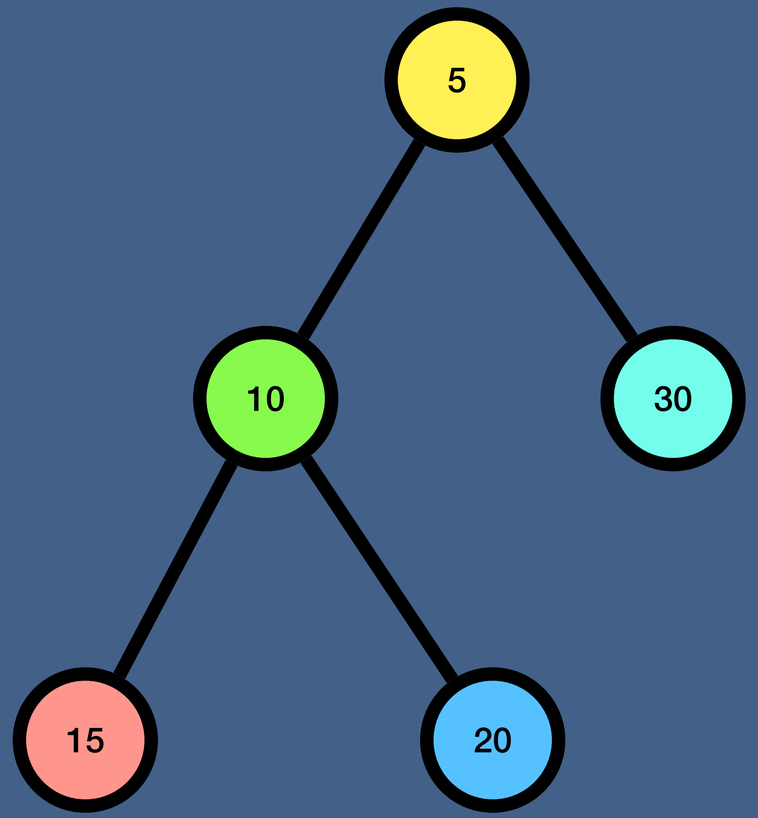
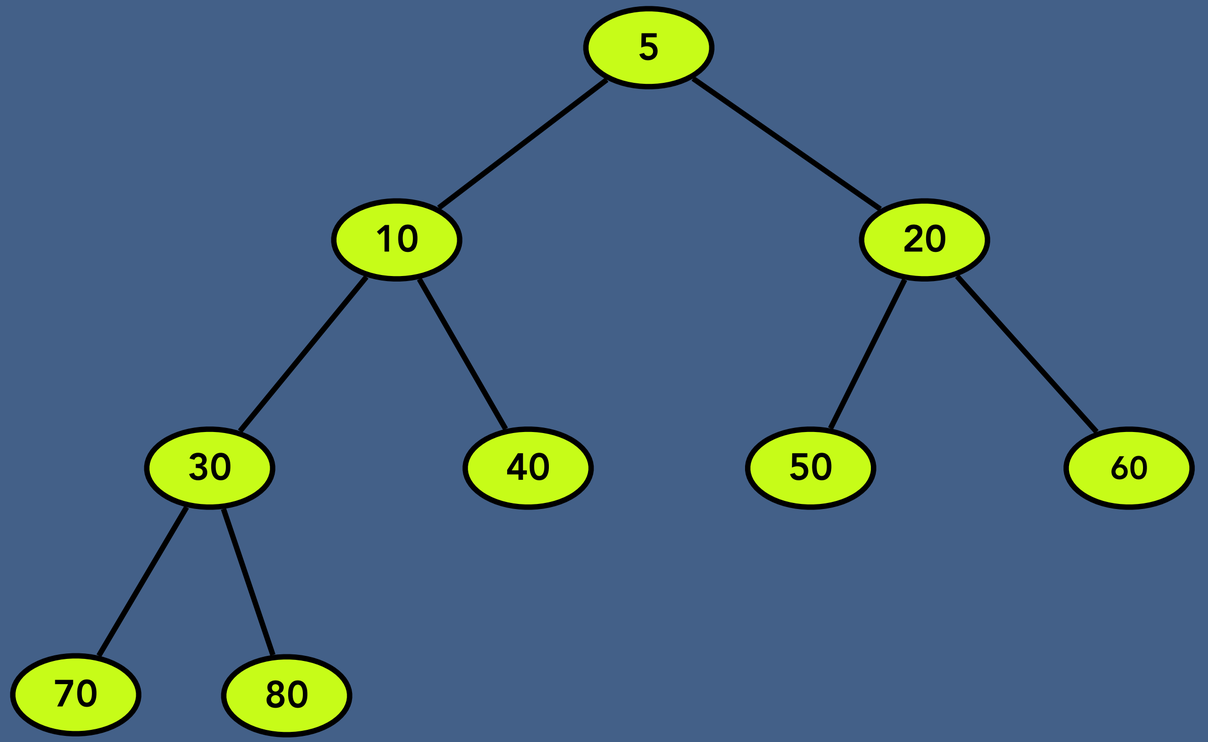
Binary Heap



What is a Binary Heap?

A Binary Heap is a Binary Tree with following properties.

- A Binary Heap is either Min Heap or Max Heap. In a Min Binary Heap, the key at root must be minimum among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree.
- It's a complete tree (All levels are completely filled except possibly the last level and the last level has all keys as left as possible). This property of Binary Heap makes them suitable to be stored in an array.

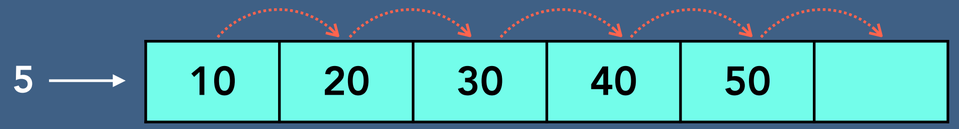


Why we need a Binary Heap?

Find the minimum or maximum number among a set of numbers in logN time. And also we want to make sure that inserting additional numbers does not take more than O(logN) time

Possible Solutions

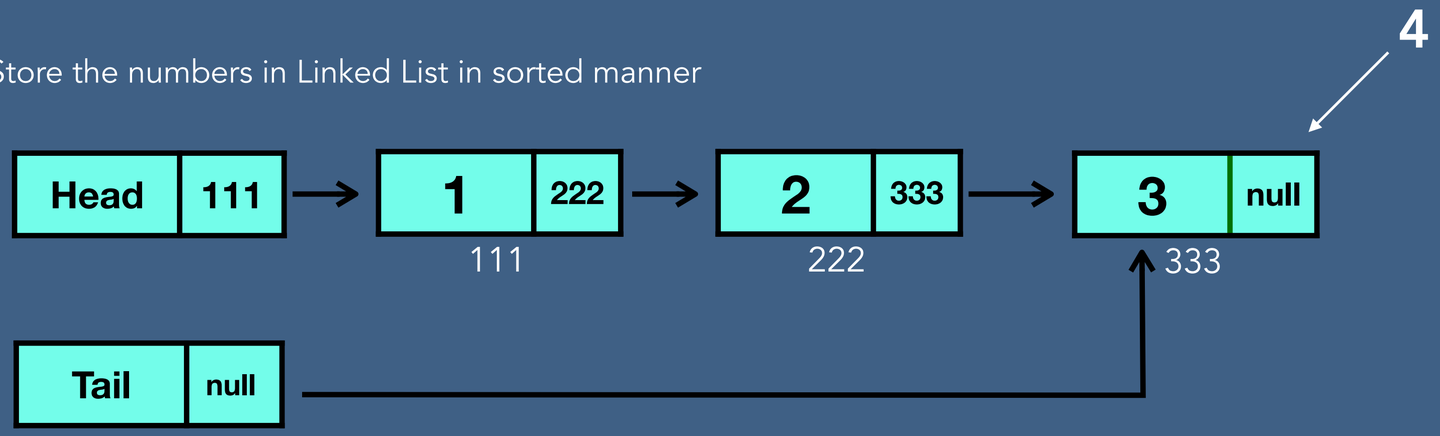
- Store the numbers in sorted Array



Find minimum: **O(1)**

Insertion: **O(n)**

- Store the numbers in Linked List in sorted manner

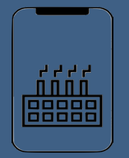


Why we need a Binary Heap?

Find the minimum or maximum number among a set of numbers in logN time. And also we want to make sure that inserting additional numbers does not take more than O(logN) time

Practical Use

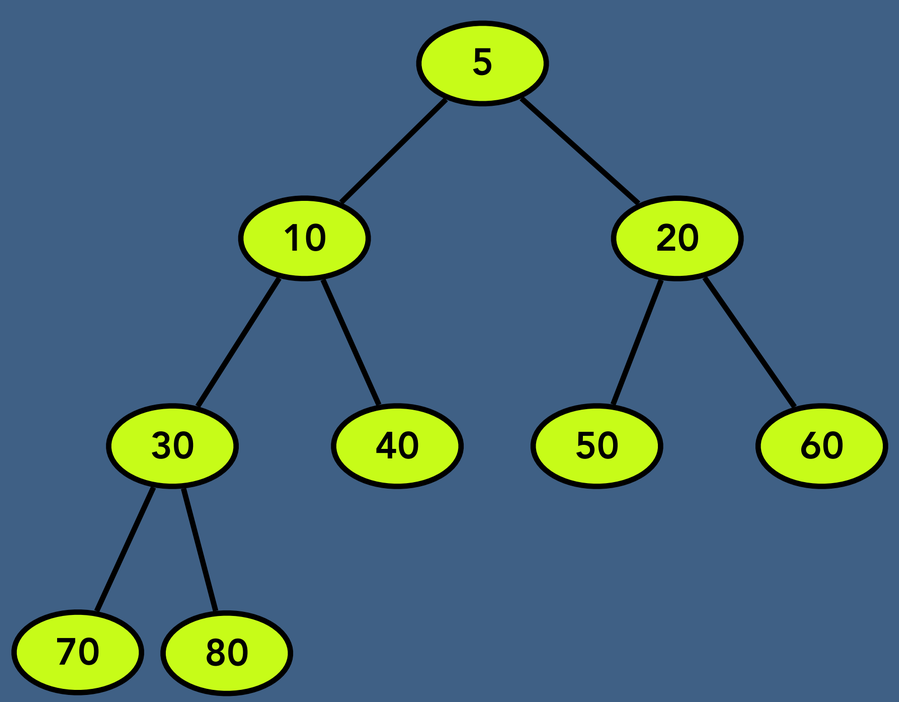
- Prim's Algorithm
- Heap Sort
- Priority Queue

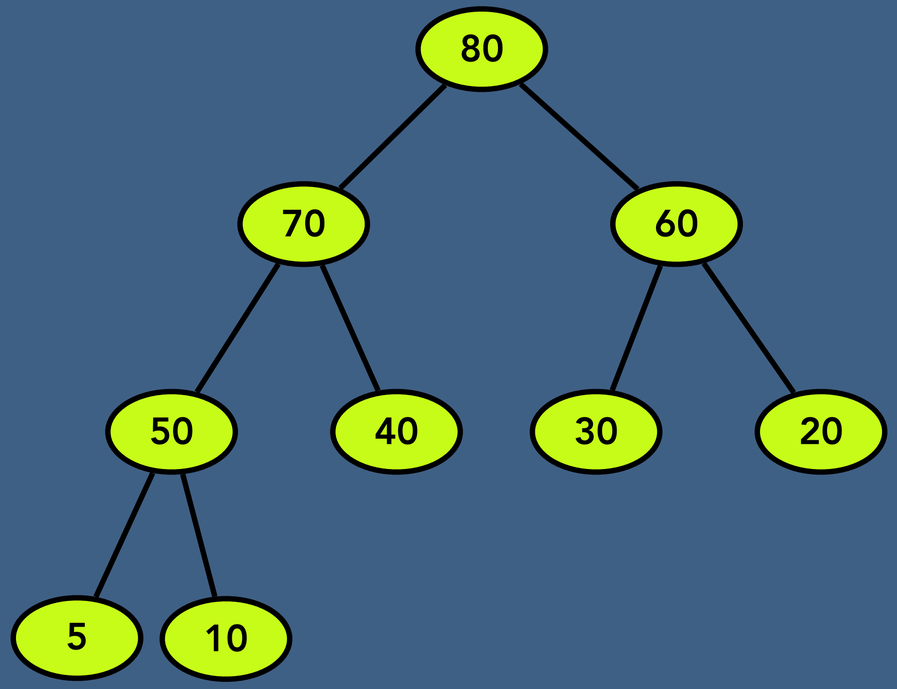


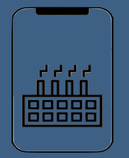
Types of Binary Heap

Min heap - the value of each node is less than or equal to the value of both its children.

Max heap - it is exactly the opposite of min heap that is the value of each node is more than or equal to the value of both its children.

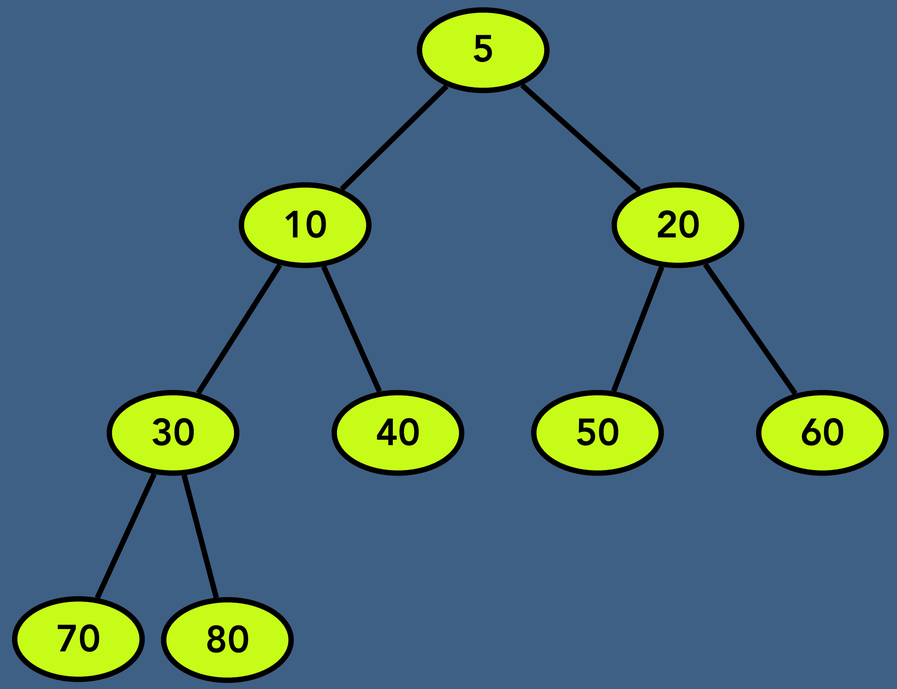






Common Operations on Binary Heap

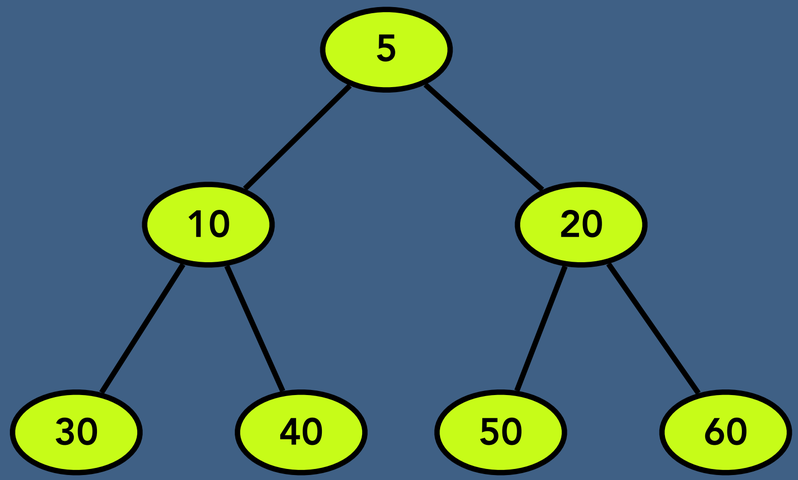
- Creation of Binary Heap,
- Peek top of Binary Heap
- Extract Min / Extract Max
- Traversal of Binary Heap
- Size of Binary Heap
- Insert value in Binary Heap
- Delete the entire Binary heap



Implementation Options

- Array Implementation
- Reference /pointer Implementation

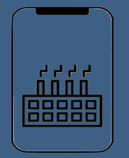
Common Operations on Binary Heap



0	1	2	3	4	5	6	7	8
✗	5	10	20	30	40	50	60	

Left child = cell[2x]

Right child = cell[2x+1]



Common Operations on Binary Heap

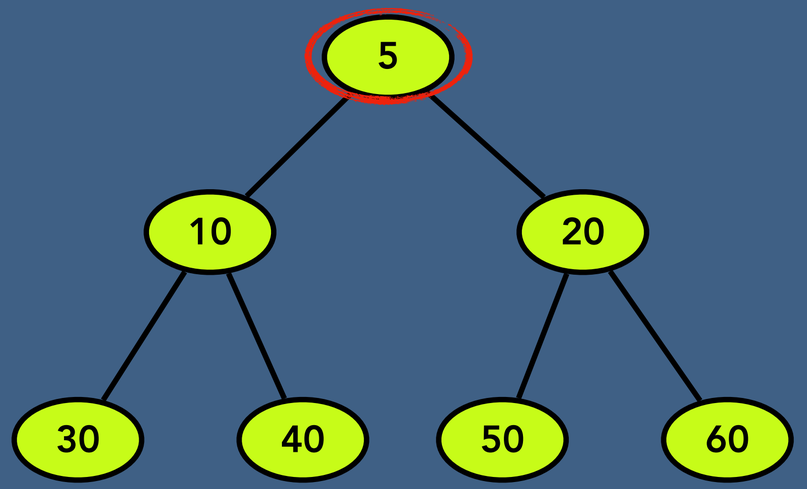
- Creation of Binary Heap

Initialize Array
set size of Binary Heap to 0

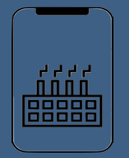
Common Operations on Binary Heap

- Peek of Binary Heap

Return Array[1]



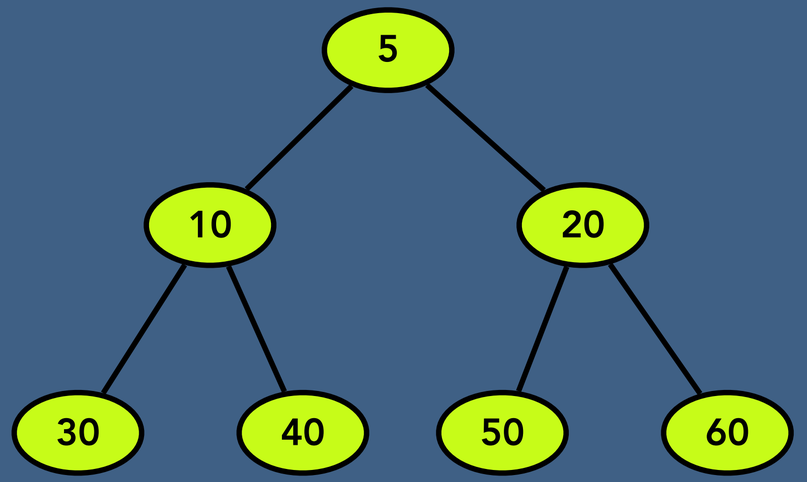
0	1	2	3	4	5	6	7	8
✗	5	10	20	30	40	50	60	



Common Operations on Binary Heap

- Size Binary Heap

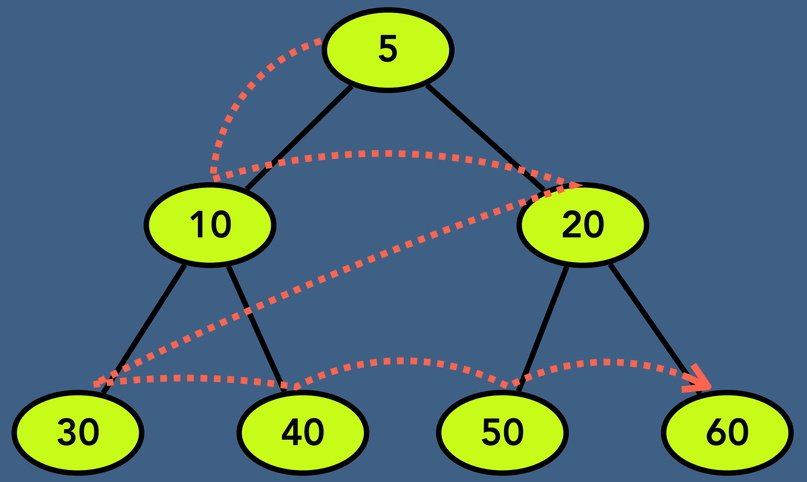
Return number of filled cells



0	1	2	3	4	5	6	7	8
✗	5	10	20	30	40	50	60	

Common Operations on Binary Heap

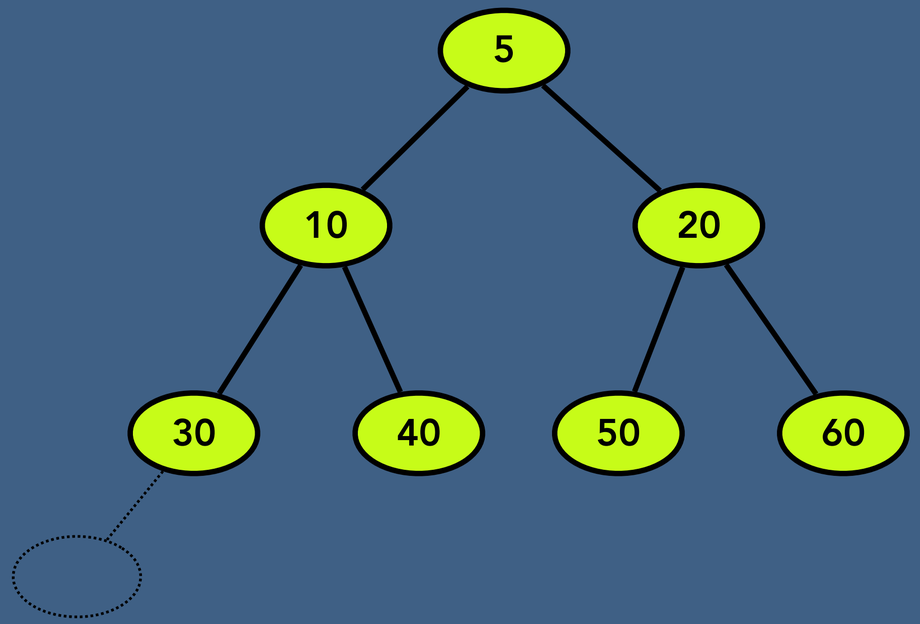
- Level Order Traversal



0	1	2	3	4	5	6	7	8
✗	5	10	20	30	40	50	60	

Binary Heap - Insert a Node

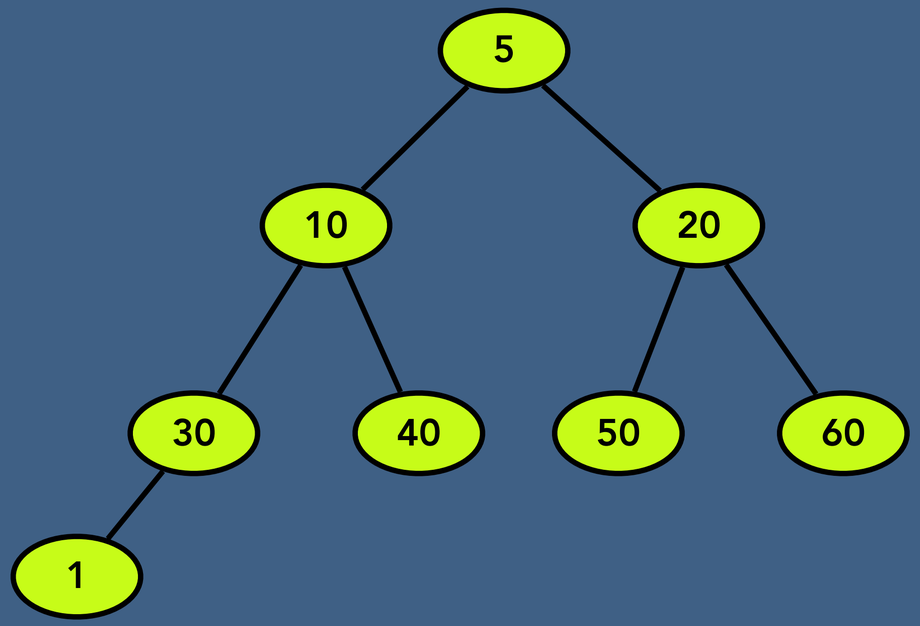
1



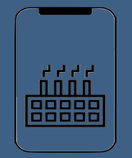
0	1	2	3	4	5	6	7	8
✕	5	10	20	30	40	50	60	

Binary Heap - Insert a Node

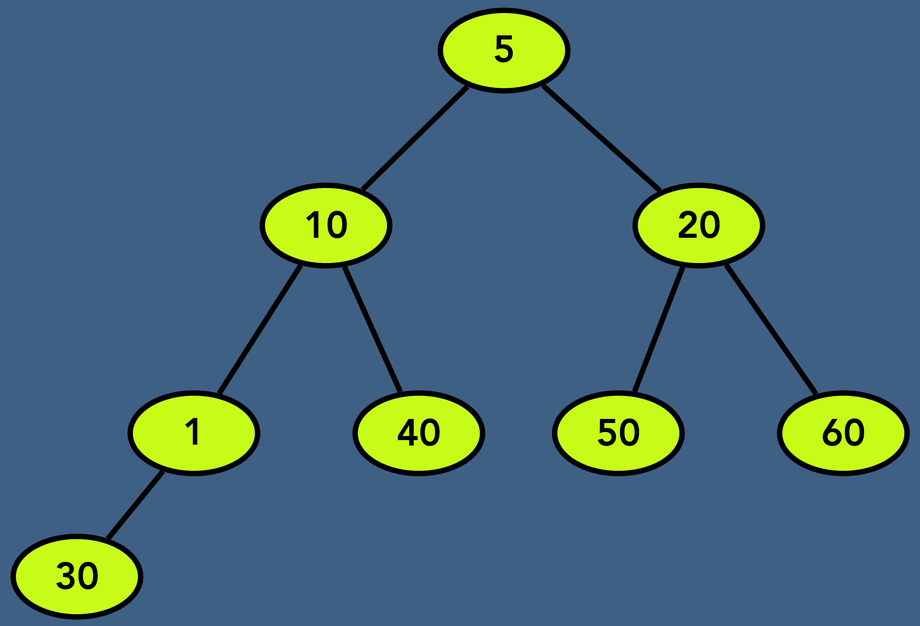
1



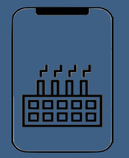
0	1	2	3	4	5	6	7	8
✗	5	10	20	1	40	50	60	30



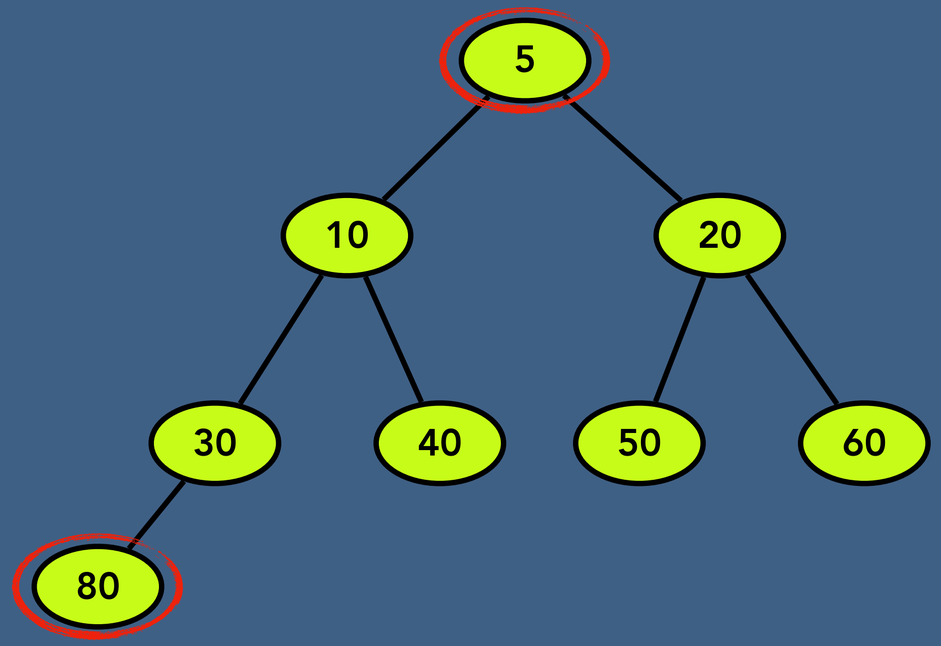
Binary Heap - Insert a Node



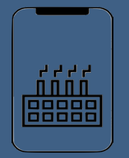
0	1	2	3	4	5	6	7	8
✗	5	5	20	10	40	50	60	30



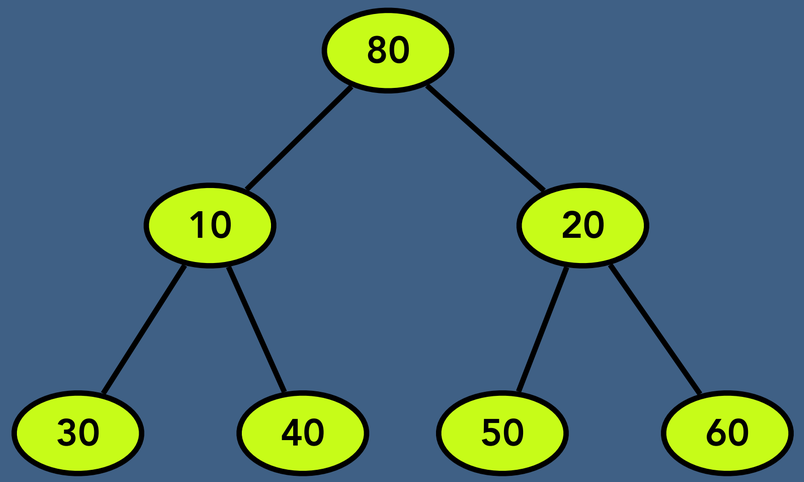
Binary Heap - Extract a Node



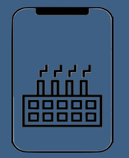
0	1	2	3	4	5	6	7	8
✕	5	10	20	30	40	50	60	80



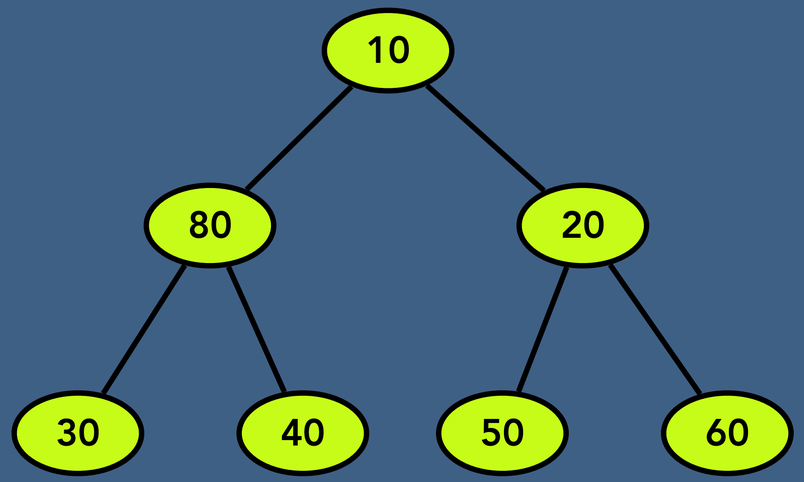
Binary Heap - Extract a Node



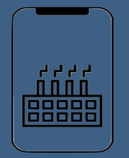
0	1	2	3	4	5	6	7	8
✕	80	10	20	30	40	50	60	



Binary Heap - Extract a Node

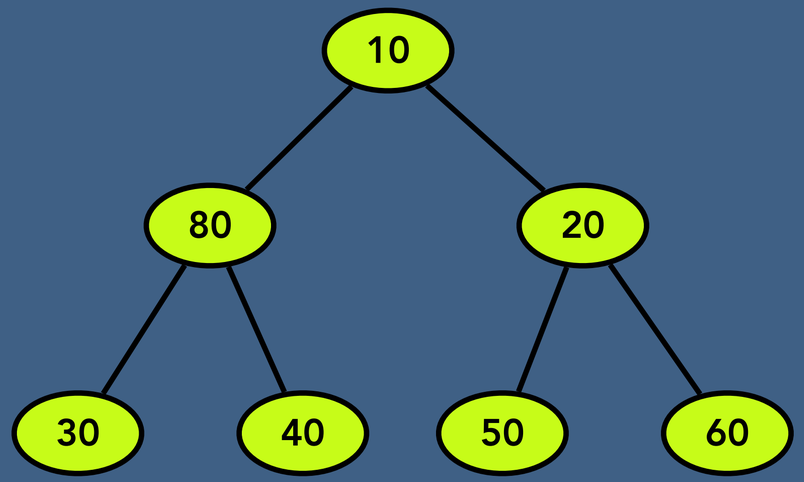


0	1	2	3	4	5	6	7	8
✕	10	80	20	30	40	50	60	

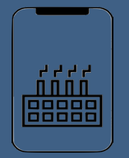


Binary Heap - Delete

array = Null



0	1	2	3	4	5	6	7	8
✗	10	80	20	30	40	50	60	



Time and Space Complexity of Binary Heap

	Time complexity	Space complexity
Create Binary Heap	O(1)	O(N)
Peek of Heap	O(1)	O(1)
Size of Heap	O(1)	O(1)
Traversal of Heap	O(N)	O(1)
Insert a node to Binary Heap	O(logN)	O(logN)
Extract a node from Binary Heap	O(logN)	O(logN)
Delete Entire Binary Heap	O(1)	O(1)